# Online-BSS spectral row sampling with optimal expected sample size

Michael B. Cohen, Cameron Musco, Jakub Pachocki

doi:10.4086/toc.2020.v016a015; CC BY 3.0

Editorial note. Complete original mathematical text, author algorithms and proofs, and original published bibliography retained. Native frontmatter/biographies remain in raw source/publication PDF assets. Existing article interface renders ToC notation for XeLaTeX only.

Editorial note. One general primary object only. Introductory restatements, source-local special cases and direct application consequences remain complete context. Exact original CC BY3.0 and model/field/parameter conditions retained.

Editorial note. Authors own mathematics; no proof review, mathematical repair, added assumptions, content hashes or cross-version comparison.

Allowed standard prerequisites (editorial).

- Original model conventions and cited standard tools. Original sharing, matrix-order or testing model, parameter restrictions and probability/error conditions retained literally.. Complete source-local core argument retained without collector reproof or independent certification.

## 1 Introduction

### 1.1 Background

A spectral approximation to a tall $n \times d$ matrix A is a smaller, typically $\tilde{\mathcal{O}}(d) \times d$ matrix $\tilde{\mathrm{A}}$ such that $\|\tilde{\mathrm{A}}\mathrm{x}\|_2 \approx \|\mathrm{A}\mathrm{x}\|_2$ for all x. Typically one asks for a multiplicative approximation, which guarantees that $(1-\epsilon)\|\mathrm{A}\mathrm{x}\|_2^2 \leq \|\tilde{\mathrm{A}}\mathrm{x}\|_2^2 \leq (1+\epsilon)\|\mathrm{A}\mathrm{x}\|_2^2$. In other notation,

$$(1-\epsilon)\mathrm{A} \preceq \tilde{\mathrm{A}} \preceq (1+\epsilon)\mathrm{A}.$$

Such approximations have many applications, most notably for solving least squares regression over A [9, 11]. If A is the vertex-edge incidence matrix of a graph, $\tilde{\mathrm{A}}$ is a spectral sparsifier [26]. It can be used to approximate effective resistances, spectral clustering, mixing time and random walk properties, and many other computations.

A number of recent papers focus on fast algorithms for spectral approximation. Using sparse random subspace embeddings [9, 23, 22], it is possible to find $\tilde{\mathrm{A}}$ in input sparsity time—i. e., running time scaling linearly in the number of nonzero entries in A. These methods produce $\tilde{\mathrm{A}}$ by randomly recombining the rows of A into a smaller number of rows. In some cases these embeddings are not enough, as it is desirable for the rows of $\tilde{\mathrm{A}}$ to be a subset of the rows of A. If A is sparse, this ensures that $\tilde{\mathrm{A}}$ is also sparse. If A represents a graph, it ensures that $\tilde{\mathrm{A}}$ is also a graph, specifically a weighted subgraph of the original.

It is well known that sampling $\mathcal{O}(d \log d/\epsilon^2)$ rows of A with probabilities proportional to their leverage scores yields a $(1 \pm \epsilon)$-factor spectral approximation to A. Further, this sampling can be done in input sparsity time, either using subspace embeddings to approximate leverage scores, or using iterative sampling techniques [20], some that only work with subsampled versions of the original matrix [11].

## 1.2 Streaming and online row sampling

When A is very large, input sparsity running times are not enough—memory restrictions also become important. Hence, recent work has tackled row sampling in a streaming model of computation. [16] presents a simple algorithm for sampling rows from an insertion-only stream, using space approximately proportional to the size of the final approximation. [15] gives a sparse-recovery based algorithm that works in dynamic streams with row insertions and deletions, also using nearly optimal space. Unfortunately, to handle dynamic streams, the algorithm in [15] is complex, requires additional restrictions on the input matrix, and uses significantly suboptimal running time to recover a spectral approximation from its low memory representation of the input stream.

While the algorithm in [16] is simple and efficient, we believe that its proof is incomplete, and do not see an obvious way to fix it. The main idea behind the algorithm is to sample rows by their leverage scores with respect to the stream seen so far. These leverage scores may be coarse overestimates of the true scores. However as more rows are streamed in, better estimates can be obtained and the sampled rows pruned to a smaller set. Unfortunately, the probability of sampling a row becomes dependent on which other rows are sampled. This seems to break the argument in that paper, which essentially claims that their process has the same distribution as would a single round of leverage score sampling.[1]

In this paper we initiate the study of row sampling in an online setting. As in an insertion stream, we read rows of A one by one. However, upon seeing a row, we immediately decide whether it should be kept in the spectral approximation or discarded, without ever retracting these decisions. We present a similar algorithm to [16], however, since we never prune previously sampled rows, the probability of sampling a row only depends on whether previous rows in the stream were sampled. This limited dependency structure allows us to rigorously argue that a spectral approximation is obtained.

In addition to addressing gaps in the literature on streaming spectral approximation, our restricted model extends work on online algorithms for a variety of other machine learning and data analysis problems, including principal component analysis [4], clustering [21], classification [3, 14], and regression [14]. In practice, online algorithms are beneficial since they can be highly computationally and memory efficient. Further, they can be applied in scenarios in which data is produced in a continuous stream and intermediate results must be output as the stream is processed. Spectral approximation is a widely applicable primitive for approximate learning and computation, so studying its implementation in an online setting is a natural direction. Since the initial publication of this work, online row sampling methods have found applications in kernel matrix approximation [7, 8] and sliding window algorithms for streaming matrix approximation [6].

## 1.3 Our results

Our primary contribution is a very simple algorithm for leverage score sampling in an online manner. The main difficultly with row sampling using leverage scores is that leverage scores themselves are not easy to compute. They are given by $l_i = \mathrm{a}_i^T(\mathrm{A}^T\mathrm{A})^{-1}\mathrm{a}_i$, and so require solving systems in $\mathrm{A}^T\mathrm{A}$ if computed naively. This is not only expensive, but also impossible in an online setting, where we do not have access to all of A.

A critical observation is that it always suffices to sample rows by overestimates of their true leverage scores. The number of rows that must be sampled is proportional to the sum of these overestimates. Since the leverage score of a row can only go up when we remove rows from the matrix, a simple way to obtain an overestimate is to compute leverage score using just a subset of the other rows of A. That is, letting $\mathrm{A}_j$ contain just $j$ of the $n$ rows of A, we can overestimate $l_i$ by $\tilde{l}_i = \mathrm{a}_i^T(\mathrm{A}_j^T\mathrm{A}_j)^{-1}\mathrm{a}_i$

[11] shows that if $\mathrm{A}_j$ is a subset of rows sampled uniformly at random, then the expected leverage score of $\mathrm{a}_i$ is $d/j$. This simple fact immediately gives a result for online sampling from a randomly ordered stream. If we compute the leverage score of the current row $\mathrm{a}_i$ against all previously seen rows (or some approximation to these rows), then the expected sum of our overestimates is bounded by $d + d/2 + \cdots + \cdots + d/n = \mathcal{O}(d \log n)$. So, sampling $\mathcal{O}(d \log d \log n/\epsilon^2)$ rows is enough obtain a $(1+\epsilon)$ multiplicative-error spectral approximation.

[1] Since the initial publication this work, the independence issue in [16] has been resolved by [18], which presents a similar algorithm for insertion-only streams that admits a correct proof.

What if we cannot guarantee a randomly ordered input stream? Is there any hope of being able to compute good leverage score estimates in an online manner? Surprisingly the answer to this is yes—we can in fact run nearly the exact same algorithm and be guaranteed that the sum of estimated leverage scores is low, regardless of stream order. Roughly, each time we receive a row which has high leverage score with respect to the previous rows, it must compose a significant part of the spectrum of A. If A does not continue to grow unboundedly, there simply cannot be too many of these significant rows.

Specifically, we show that if we sample by the ridge leverage scores [1] over all previously seen rows, which are the leverage scores computed over $\mathrm{A}_i^T\mathrm{A}_i + \lambda\mathrm{I}$ for some small regularizing factor $\lambda$, then with just $\mathcal{O}(d\log d\log(\epsilon\|\mathrm{A}\|_2^2/\delta)/\epsilon^2)$ samples we obtain a $(1+\epsilon)$ multiplicative-error, $\delta$ additive-error spectral approximation. That is, with high probability we sample a matrix $\tilde{\mathrm{A}}$ with $(1-\epsilon)\mathrm{A}^T\mathrm{A} - \delta\mathrm{I} \preceq \tilde{\mathrm{A}}^T\tilde{\mathrm{A}} \preceq (1+\epsilon)\mathrm{A}^T\mathrm{A} + \delta\mathrm{I}$.

To gain intuition behind this bound, note that we can convert it into a multiplicative one by setting $\delta = \epsilon\sigma_{min}(\mathrm{A})^2$ where $\sigma_{min}(\mathrm{A})$ is the minimum singular value of A (as long as we have some estimate of $\sigma_{min}(\mathrm{A})$). This setting of $\delta$ will require taking $\mathcal{O}(d\log d\log(\kappa(\mathrm{A}))/\epsilon^2)$ samples, where $\kappa(\mathrm{A}) = \sigma_{max}(\mathrm{A})/\sigma_{min}(\mathrm{A})$ is the condition number of A. If we have a polynomial bound on this condition number, as we do, for instance, for graphs with polynomially bounded edge weights, this becomes $\mathcal{O}(d\log^2 d/\epsilon^2)$—nearly matching the $\mathcal{O}(d\log d/\epsilon^2)$ achievable if sampling by true leverage scores.

Our online sampling algorithm is extremely simple. When each row comes in, we compute the online ridge leverage score, or an estimate of it, and then irrevocably either add the row to our approximation or remove it. As mentioned, it is similar in form to the streaming algorithm of [16], except that it does not require pruning previously sampled rows. This allows us to avoid difficult dependency issues. Additionally, without pruning, we do not even need to store all previously sampled rows. As long as we store a constant-factor spectral approximation our previous samples, we can compute good approximations to the online ridge leverage scores. In this way, we can store just $\mathcal{O}(d\log d\log(\epsilon\|\mathrm{A}\|_2^2/\delta))$ rows in working memory ($\mathcal{O}(d\log^2 d)$ if we want a spectral graph sparsifier), filtering our input stream into an $\mathcal{O}(d\log d\log(\kappa(\mathrm{A}))/\epsilon^2)$-size output stream. Note that this memory bound in fact improves as $\epsilon$ decreases, and regardless, can be significantly smaller than the output size of the algorithm.

In addition to our main sampling result, we use our bounds on online ridge leverage score approximations to show that an algorithm in the style of [2] allows us to remove a $\log d$ factor and sample just $\mathcal{O}(d\log(\epsilon\|\mathrm{A}\|_2^2/\delta)/\epsilon^2)$ rows (Theorem 4.1). This algorithm is more complex and can require $\mathcal{O}(d^2)$ working memory. However, in Theorem 5.1 we show that it is asymptotically optimal. The $\log(\epsilon\|\mathrm{A}\|_2^2/\delta)$ factor is not an artifact of our analysis, but is truly the cost of the restricting ourselves to online sampling. No algorithm can obtain a multiplicative $(1+\epsilon)$ additive $\delta$ spectral approximation taking fewer than $\Omega(d\log(\epsilon\|\mathrm{A}\|_2^2/\delta)/\epsilon^2)$ rows in an online manner.

## 2 Overview

Let A be an $n \times d$ matrix with rows $\mathrm{a}_1, \ldots, \mathrm{a}_n$. A natural approach to row sampling from A is picking an a priori probability with which each row is kept, and then deciding whether to keep each row independently. A common choice is for the sampling probabilities to be proportional to the leverage scores of the rows. The leverage score of the $i$-th row of A is defined to be

$$\mathrm{a}_i^T(\mathrm{A}^T\mathrm{A})^\dagger\mathrm{a}_i\,,$$

where the dagger symbol denotes the pseudoinverse. In this work, we will be interested in approximating $\mathrm{A}^T\mathrm{A}$ with some (very) small multiple of the identity added. Hence, we will be interested in the $\lambda$-ridge leverage scores [1]:

$$\mathrm{a}_i^T(\mathrm{A}^T\mathrm{A} + \lambda\mathrm{I})^{-1}\mathrm{a}_i\,,$$

for a parameter $\lambda > 0$.

In many applications, obtaining the (nearly) exact values of $\mathrm{a}_i^T(\mathrm{A}^T\mathrm{A}+\lambda\mathrm{I})^{-1}\mathrm{a}_i$ for sampling is difficult or outright impossible. A key idea is that as long as we have a sequence $l_1, \ldots, l_n$ of overestimates of the $\lambda$-ridge leverage scores, that is, for $i = 1, \ldots, n$,

$$l_i \geq \mathrm{a}_i^T(\mathrm{A}^T\mathrm{A} + \lambda\mathrm{I})^{-1}\mathrm{a}_i\,,$$

we can sample by these overestimates and obtain rigorous guarantees on the quality of the obtained spectral approximation. This notion is formalized in Theorem 2.1.

Theorem 2.1. Let A be an $n \times d$ matrix with rows $\mathrm{a}_1, \ldots, \mathrm{a}_n$. Let $\epsilon \in (0,1), \delta > 0, \lambda := \delta/\epsilon, c := 8 \log d/\epsilon^2$. Assume we are given $l_1, \ldots, l_n$ such that for all $i = 1, \ldots, n$,

$$l_i \geq \mathrm{a}_i^T (\mathrm{A}^T\mathrm{A} + \lambda \mathrm{I})^{-1} \mathrm{a}_i \,.$$

For $i = 1, \ldots, n$, let $p_i := \min(cl_i, 1)$. Construct $\mathbb{A}$ by independently sampling each row $\mathrm{a}_i$ of A with probability $p_i$, and rescaling it by $1/\sqrt{p_i}$ if it is included in the sample. Then, with high probability,

$$(1-\epsilon)\mathrm{A}^T\mathrm{A} - \delta \mathrm{I} \preceq \mathbb{A}^T\mathbb{A} \preceq (1+\epsilon)\mathrm{A}^T\mathrm{A} + \delta \mathrm{I},$$

and the number of rows in $\mathbb{A}$ is $\mathcal{O}\left((\sum_{i=1}^n l_i) \log d/\epsilon^2\right)$.

Proof. This sort of guarantee for leverage score sampling is well known. See for example Lemma 4 of [11]. If we sampled both the rows of A and the rows of $\sqrt{\lambda}\mathrm{I}$ with the leverage scores over $(\mathrm{A}^T\mathrm{A} + \lambda\mathrm{I})$, we would have $(1-\epsilon)(\mathrm{A}^T\mathrm{A} + \lambda\mathrm{I}) \preceq \mathbb{A}^T\mathbb{A} \preceq (1+\epsilon)(\mathrm{A}^T\mathrm{A} + \lambda\mathrm{I})$. However, we do not sample the rows of the identity. Since we could have sampled them each with probability 1, we can simply subtract $\lambda\mathrm{I} = (\delta/\epsilon)\mathrm{I}$ from the multiplicative bound and have $(1-\epsilon)\mathrm{A}^T\mathrm{A} - \delta\mathrm{I} \preceq \mathbb{A}^T\mathbb{A} \preceq (1+\epsilon)\mathrm{A}^T\mathrm{A} + \delta\mathrm{I}$. □

The idea of using overestimates of leverage scores to perform row sampling has been applied successfully to various problems (see, e. g., [17, 11]). However, in these applications, access to the entire matrix is required beforehand. In the streaming and online settings, we have to rely on partial data to approximate the true leverage scores. The most natural idea is to just use the portion of the matrix seen thus far as an approximation to A. This leads us to introduce the online $\lambda$-ridge leverage scores:

$$l_i := \min(\mathrm{a}_i^T (\mathrm{A}_{i-1}^T \mathrm{A}_{i-1} + \lambda \mathrm{I})^{-1} \mathrm{a}_i \,, 1),$$

where $\mathrm{A}_i$ $(i = 0, \ldots, n)$ is defined as the matrix consisting of the first $i$ rows of A.[2]

Since clearly $\mathrm{A}_i^T\mathrm{A}_i \preceq \mathrm{A}^T\mathrm{A}$ for all $i$, it is not hard to see that $l_i$ does overestimate the true $\lambda$-ridge leverage score for row $\mathrm{a}_i$. A more complex question, however, is establishing an upper bound on $\sum_{i=1}^n l_i$ so that we can bound the number of samples needed by Theorem 2.1.

A core result of this work, stated in Theorem 2.2, is establishing such an upper bound; in fact, this bound is shown to be tight up to constants (Theorem 5.1) and is nearly linear in most cases.

Theorem 2.2. Let A be an $n \times d$ matrix with rows $\mathrm{a}_1, \ldots, \mathrm{a}_n$. Let $\mathrm{A}_i$ for $i \in \{0, \ldots, n\}$ be the matrix consisting of the first $i$ rows of A. For $\lambda > 0$, let

$$l_i := \min(\mathrm{a}_i^T (\mathrm{A}_{i-1}^T \mathrm{A}_{i-1} + \lambda \mathrm{I})^{-1} \mathrm{a}_i \,, 1)$$

be the online $\lambda$-ridge leverage score of the $i^{th}$ row of A. Then

$$\sum_{i=1}^n l_i = \mathcal{O}(d \log(\|\mathrm{A}\|_2^2/\lambda)).$$

Theorems 2.1 and 2.2 suggest a simple algorithm for online row sampling: simply use the online $\lambda$-ridge leverage scores, for $\lambda := \delta/\epsilon$. This gives a spectral approximation with $\mathcal{O}(d \log d \log(\epsilon\|\mathrm{A}\|_2^2/\delta)/\epsilon^2)$ rows. Unfortunately, computing each $l_i$ exactly requires us to store all the rows we have seen in memory (or alternatively to store the sum of their outer products, $\mathrm{A}_i^T\mathrm{A}_i$). In many cases, such a requirement would defeat the purpose of streaming row sampling.

A natural idea is to use the sample we have kept thus far as an approximation to $\mathrm{A}_i$ when computing $l_i$. It turns out that the approximate online ridge leverage scores $\tilde{l}_i$ computed in this way will not always be good approximations to $l_i$; however, we can still prove that they satisfy the requisite bounds and yield the same row sample size! We formalize these results in the algorithm Online-Sample (Figure 1) and Theorem 2.3.

[2] We use the proposed scores $l_i$ for simplicity, however note that the following, perhaps more natural, definition of online leverage scores would also be effective: $l_i' := \mathrm{a}_i^T (\mathrm{A}_i^T \mathrm{A}_i + \lambda \mathrm{I})^{-1} \mathrm{a}_i$ .

$\mathbb{A} =$ Online-Sample$(\mathrm{A}, \epsilon, \delta)$, where A is an $n \times d$ matrix with rows $\mathrm{a}_1, \ldots, \mathrm{a}_n$, $\epsilon \in (0,1)$, $\delta > 0$.

1. Set $\lambda := \delta/\epsilon$, $c := 8 \log d/\epsilon^2$.
2. Let $\mathbb{A}_0$ be a $0 \times d$ matrix.
3. For $i = 1, \ldots, n$:
   (a) Let $\tilde{l}_i := \min((1+\epsilon)\mathrm{a}_i^T(\mathbb{A}_{i-1}^T\mathbb{A}_{i-1} + \lambda \mathrm{I})^{-1}\mathrm{a}_i\,, 1)$.
   (b) Let $p_i := \min(c\tilde{l}_i\,, 1)$.
   (c) Set $\mathbb{A}_i := \begin{cases} \begin{bmatrix} \mathbb{A}_{i-1} \\ \mathrm{a}_i/\sqrt{p_i} \end{bmatrix} & \text{with probability } p_i, \\ \mathbb{A}_{i-1} & \text{otherwise.} \end{cases}$
4. Return $\mathbb{A} := \mathbb{A}_n$.

Figure 1: The basic online sampling algorithm

Theorem 2.3. Let $\mathbb{A}$ be the matrix returned by Online-Sample$(\mathrm{A}, \epsilon, \delta)$. With high probability,

$$(1-\epsilon)\mathrm{A}^T\mathrm{A} - \delta\mathrm{I} \preceq \mathbb{A}^T\mathbb{A} \preceq (1+\epsilon)\mathrm{A}^T\mathrm{A} + \delta\mathrm{I},$$

and the number of rows in $\mathbb{A}$ is $\mathcal{O}(d \log d \log(\epsilon\|\mathrm{A}\|_2^2/\delta)/\epsilon^2)$.

To save computation, we note that, with a small modification, we can run Online-Sample with batch processing of rows. Specifically, say we start from the $i^{th}$ position in the stream. we can store the next $b = \mathcal{O}(d)$ rows. We can then compute sampling probabilities for these rows all at once using a system solver for $(\mathbb{A}_{i+b}^T\mathbb{A}_{i+b} + \lambda\mathrm{I})$. Using a trick introduced in [25], by applying a Johnson–Lindenstrauss random projection to the rows whose scores we are computing, we need just $\mathcal{O}(\log(1/\gamma))$ system solves to compute constant-factor approximations to the ridge scores with probability $1-\gamma$. If we set $\gamma = 1/\mathrm{poly}(n)$ then we can union bound over our whole stream, using this trick with each batch of $\mathcal{O}(d)$ input rows. The batch probabilities will only be closer to the true ridge leverage scores than the non-batch probabilities and we will enjoy the same guarantees as Online-Sample.

Additionally, it turns out that with a simple trick, it is possible to reduce the memory usage of the algorithm by a factor of $\epsilon^{-2}$, bringing it down to $\mathcal{O}(d \log d \log(\epsilon\|\mathrm{A}\|_2^2/\delta))$ (assuming the row sample is output to an output stream). Note that this expression gets smaller with $\epsilon$; hence we obtain a row sampling algorithm with memory complexity independent of desired multiplicative precision. The basic idea is that, instead of keeping all previously sampled rows in memory, we store a smaller set of rows that give a constant-factor spectral approximation, still enough to give good estimates of the online ridge leverage scores.

This result is presented in the algorithm Slim-Sample (Figure 2) and Lemma 3.5. A particularly interesting consequence for graphs with polynomially bounded edge weights is

Corollary 2.4. Let $G$ be a simple graph on $d$ vertices, and $\epsilon \in (0,1)$. We can construct a $(1+\epsilon)$-sparsifier of $G$ of size $\mathcal{O}(d \log^2 d/\epsilon^2)$, using only $\mathcal{O}(d \log^2 d)$ working memory in the online model.

Proof. Let $\sigma_{min}(\mathrm{A})$ and $\sigma_{max}(\mathrm{A}) = \|\mathrm{A}\|_2$ be the smallest and largest singular values of A respectively. Let $\kappa(\mathrm{A}) = \sigma_{max}(\mathrm{A})/\sigma_{min}(\mathrm{A})$ be the condition number of A. If we apply Theorem 2.3 with $\delta = \epsilon/\sigma_{min}^2(\mathrm{A})$. we require sample complexity $O(d \log d \log(\epsilon\|\mathrm{A}\|_2^2/\delta)/\epsilon^2) = O(d \log d \log(\kappa(\mathrm{A})^2)/\epsilon^2)$. For an unweighted graph on $d$ vertices, $\sigma_{max}(\mathrm{A})^2 \leq d$, since $d$ is the largest squared singular value of the complete graph. Combining with Lemma 6.1 of [27], we have that the condition number of a graph on $d$ vertices whose edge weights are within a multiplicative $\mathrm{poly}(d)$ of each other is polynomial in $d$. So $\log(\kappa^2(\mathrm{A})) = \mathcal{O}(\log d)$, which gives the corollary. □

We remark that the algorithm of Corollary 2.4 can be made to run in nearly linear time in the stream size. We combine Slim-Sample with the batch processing idea described above. Because A

is a graph, our matrix approximation is always a symmetric diagonally dominant matrix, with $\mathcal{O}(d)$ nonzero entries. We can solve systems in it in time $\tilde{\mathcal{O}}(d)$. Using the Johnson–Lindenstrauss random projection trick of [25], we can compute approximate ridge leverage scores for a batch of $\mathcal{O}(d)$ rows with failure probability polynomially small in $n$ in $\tilde{\mathcal{O}}(d \log n)$ time. Union bounding over the whole stream, we obtain nearly linear running time.

To complement the row sampling results discussed above, we explore the limits of the proposed online setting. In Section 4 we present the algorithm Online-BSS, which obtains spectral approximations with $\mathcal{O}(d \log(\epsilon \|\mathrm{A}\|_2^2/\delta)/\epsilon^2)$ rows in the online setting (with larger memory requirements than the simpler sampling algorithms). Its analysis is given in Theorem 4.1. In Section 5, we show that this number of samples is in fact the best achievable, up to constant factors (Theorem 5.1). The $\log(\epsilon \|\mathrm{A}\|_2^2/\delta)$ factor is truly the cost of requiring rows to be selected in an online manner.

## 3 Analysis of sampling schemes

We begin by bounding the sum of online $\lambda$-ridge leverage scores. The intuition behind the proof of Theorem 2.2 is that whenever we add a row with a large online leverage score to a matrix, we increase its determinant significantly, as follows from the matrix determinant lemma (Lemma 3.1). Thus we can reduce upper bounding the online leverage scores to bounding the matrix determinant.

Lemma 3.1 (Matrix determinant lemma). Assume S is an invertible square matrix and u is a vector. Then

$$\det(\mathrm{S} + \mathrm{u}\mathrm{u}^T) = (\det \mathrm{S})(1 + \mathrm{u}^T \mathrm{S}^{-1} \mathrm{u}).$$

Proof of Theorem 2.2. By Lemma 3.1, we have

$$\begin{aligned}
\det(\mathrm{A}_{i+1}^T \mathrm{A}_{i+1} + \lambda \mathrm{I}) &= \det(\mathrm{A}_i^T \mathrm{A}_i + \lambda \mathrm{I}) \cdot \left(1 + \mathrm{a}_{i+1}^T (\mathrm{A}_i^T \mathrm{A}_i + \lambda \mathrm{I})^{-1} \mathrm{a}_{i+1}\right) \\
&\geq \det(\mathrm{A}_i^T \mathrm{A}_i + \lambda \mathrm{I}) \cdot (1 + l_{i+1}) \\
&\geq \det(\mathrm{A}_i^T \mathrm{A}_i + \lambda \mathrm{I}) \cdot \mathrm{e}^{l_{i+1}/2}.
\end{aligned}$$

Hence,

$$\begin{aligned}
\det(\mathrm{A}^T \mathrm{A} + \lambda \mathrm{I}) &= \det(\mathrm{A}_n^T \mathrm{A}_n + \lambda \mathrm{I}) \\
&\geq \det(\lambda \mathrm{I}) \cdot \mathrm{e}^{\sum l_i/2} \\
&= \lambda^d \mathrm{e}^{\sum l_i/2}.
\end{aligned}$$

We have $\det(\mathrm{A}^T \mathrm{A} + \lambda \mathrm{I}) \leq (\|\mathrm{A}\|_2^2 + \lambda)^d$. Therefore

$$(\|\mathrm{A}\|_2^2 + \lambda)^d \geq \lambda^d \mathrm{e}^{\sum l_i/2}.$$

Taking logarithms of both sides, we obtain

$$\begin{aligned}
d \log(\|\mathrm{A}\|_2^2 + \lambda) &\geq d \log \lambda + \sum l_i/2 \\
\sum l_i &\leq 2d \log(1 + \|\mathrm{A}\|_2^2/\lambda).
\end{aligned}$$

□

We now turn to analyzing the algorithm Online-Sample. Because the samples taken by the algorithm are not independent, we are not able to use a standard matrix Chernoff bound like the one in Theorem 2.1. However, we do know that whether we take row $i$ does not depend on later rows; thus, we are able to analyze the process as a martingale. We will use a matrix version of the Freedman inequality given by Tropp.

Theorem 3.2 (Matrix Freedman inequality [28]). Let $\mathrm{Y}_0, \mathrm{Y}_1, \ldots, \mathrm{Y}_n$ be a matrix martingale whose values are self-adjoint matrices with dimension $d$, and let $\mathrm{X}_1, \ldots, \mathrm{X}_n$ be the difference sequence. Assume that the difference sequence is uniformly bounded in the sense that

$$\|\mathrm{X}_k\|_2 \leq R \text{ almost surely, for } k = 1, \ldots, n.$$

Define the predictable quadratic variation process of the martingale as

$$\mathrm{W}_k := \sum_{j=1}^{k} \mathbb{E}\left[\mathrm{X}_j^2 \mid \mathrm{Y}_{j-1}, \ldots, \mathrm{Y}_0\right], \text{ for } k = 1, \ldots, n.$$

Then, for all $\epsilon > 0$ and $\sigma^2 > 0$,

$$P\left[\|\mathrm{Y}_n\|_2 \geq \epsilon \text{ and } \|\mathrm{W}_n\|_2 \leq \sigma^2\right] \leq d \cdot \exp\left(-\frac{\epsilon^2/2}{\sigma^2 + R\epsilon/3}\right).$$

We begin by showing that the output of Online-Sample is in fact an approximation of A, and that the approximate online leverage scores are lower bounded by the actual online leverage scores.

Lemma 3.3. After running Online-Sample, it holds with high probability that

$$(1-\epsilon)\mathrm{A}^T\mathrm{A} - \delta\mathrm{I} \preceq \mathbb{A}^T\mathbb{A} \preceq (1+\epsilon)\mathrm{A}^T\mathrm{A} + \delta\mathrm{I},$$

and also

$$\tilde{l}_i \geq \mathrm{a}_i^T(\mathrm{A}^T\mathrm{A} + \lambda\mathrm{I})^{-1}\mathrm{a}_i$$

for $i = 1, \ldots, n$.

Proof. Let

$$\mathrm{u}_i := (\mathrm{A}^T\mathrm{A} + \lambda\mathrm{I})^{-1/2}\mathrm{a}_i\,.$$

We construct a matrix martingale $\mathrm{Y}_0, \mathrm{Y}_1, \ldots, \mathrm{Y}_n \in \mathbb{R}^{d\times d}$ with the difference sequence $\mathrm{X}_1, \ldots, \mathrm{X}_n$. Set $\mathrm{Y}_0 = 0$. If $\|\mathrm{Y}_{i-1}\|_2 \geq \epsilon$, we set $\mathrm{X}_i := 0$. Otherwise, let

$$\mathrm{X}_i := \begin{cases} (1/p_i - 1)\mathrm{u}_i\mathrm{u}_i^T & \text{if } \mathrm{a}_i \text{ is sampled in } \mathbb{A}, \\ -\mathrm{u}_i\mathrm{u}_i^T & \text{otherwise.} \end{cases}$$

In the case that $\|\mathrm{Y}_{i-1}\|_2 < \epsilon$, by construction, $\|\mathrm{Y}_j\|_2 < \epsilon$ for all $j < i-1$. So we have

$$\mathrm{Y}_{i-1} = (\mathrm{A}^T\mathrm{A} + \lambda\mathrm{I})^{-1/2}(\mathbb{A}_{i-1}^T\mathbb{A}_{i-1} - \mathrm{A}_{i-1}^T\mathrm{A}_{i-1})(\mathrm{A}^T\mathrm{A} + \lambda\mathrm{I})^{-1/2}.$$

Since $\|\mathrm{Y}_{i-1}\|_2 \leq \epsilon$, we can see that

$$(\mathrm{A}^T\mathrm{A} + \lambda\mathrm{I})^{-1/2}(\mathbb{A}_{i-1}^T\mathbb{A}_{i-1})(\mathrm{A}^T\mathrm{A} + \lambda\mathrm{I})^{-1/2} \preceq (\mathrm{A}^T\mathrm{A} + \lambda\mathrm{I})^{-1/2}(\mathrm{A}_{i-1}^T\mathrm{A}_{i-1})(\mathrm{A}^T\mathrm{A} + \lambda\mathrm{I})^{-1/2} + \epsilon\mathrm{I}.$$

Multiplying on both right and left by $(\mathrm{A}^T\mathrm{A} + \lambda\mathrm{I})^{1/2}$ gives

$$\mathbb{A}_{i-1}^T\mathbb{A}_{i-1} \preceq \mathrm{A}_{i-1}^T\mathrm{A}_{i-1} + \epsilon(\mathrm{A}^T\mathrm{A} + \lambda\mathrm{I}).$$

Hence, we have

$$\begin{aligned}
\tilde{l}_i &= \min((1+\epsilon)\mathrm{a}_i^T(\mathbb{A}_{i-1}^T\mathbb{A}_{i-1} + \lambda\mathrm{I})^{-1}\mathrm{a}_i\,, 1) \\
&\geq \min((1+\epsilon)\mathrm{a}_i^T(\mathrm{A}_{i-1}^T\mathrm{A}_{i-1} + \lambda\mathrm{I} + \epsilon(\mathrm{A}^T\mathrm{A} + \lambda\mathrm{I}))^{-1}\mathrm{a}_i\,, 1) \\
&\geq \min((1+\epsilon)\mathrm{a}_i^T((1+\epsilon)(\mathrm{A}^T\mathrm{A} + \lambda\mathrm{I}))^{-1}\mathrm{a}_i\,, 1) \\
&= \mathrm{a}_i^T(\mathrm{A}^T\mathrm{A} + \lambda\mathrm{I})^{-1}\mathrm{a}_i \qquad (3.1) \\
&= \mathrm{u}_i^T\mathrm{u}_i\,.
\end{aligned}$$

Thus, $p_i = \min(c\tilde{l}_i\,, 1) \geq \min(c\mathrm{u}_i^T\mathrm{u}_i\,, 1)$. If $p_i = 1$, then $\mathrm{X}_i = 0$. Otherwise, we have $p_i \geq c\mathrm{u}_i^T\mathrm{u}_i$ and

$$\|\mathrm{X}_i\|_2 \leq \max\{1, 1/p_i - 1\} \cdot \|\mathrm{u}_i\mathrm{u}_i^T\|_2 \leq \mathrm{u}_i^T\mathrm{u}_i/p_i \leq 1/c. \qquad (3.2)$$

Further

$$\begin{aligned}\mathbb{E}\left[\mathrm{X}_i^2 \mid \mathrm{Y}_{i-1},\ldots,\mathrm{Y}_0\right] &\preceq p_i\cdot(1/p_i-1)^2(\mathrm{u}_i\mathrm{u}_i^T)^2+(1-p_i)\cdot(\mathrm{u}_i\mathrm{u}_i^T)^2\\ &= (\mathrm{u}_i\mathrm{u}_i^T)^2\cdot(1-p_i)/p_i\\ &\preceq \mathrm{u}_i\mathrm{u}_i^T\cdot\left(\mathrm{u}_i^T\mathrm{u}_i/p_i\right)\\ &\preceq \mathrm{u}_i\mathrm{u}_i^T/c. \qquad \text{(by equation (3.2))}\end{aligned}$$

And so, for the predictable quadratic variation process of the martingale $\{\mathrm{Y}_i\}$

$$\mathrm{W}_i := \sum_{k=1}^{i}\mathbb{E}\left[\mathrm{X}_k^2 \mid \mathrm{Y}_{k-1},\ldots,\mathrm{Y}_0\right],$$

we have

$$\|\mathrm{W}_i\|_2 \le \left\|\sum_{k=1}^{i}\mathrm{u}_k\mathrm{u}_k^T/c\right\|_2 \le 1/c.$$

Therefore by, Theorem 3.2, we have

$$\begin{aligned}P\left[\|\mathrm{Y}_n\|_2\ge\epsilon\right] &\le d\cdot\exp\left(\frac{-\epsilon^2/2}{1/c+\epsilon/(3c)}\right)\\ &\le d\cdot\exp(-c\epsilon^2/4)\\ &= 1/d.\end{aligned}$$

This implies that with high probability

$$\|(\mathrm{A}^T\mathrm{A}+\lambda\mathrm{I})^{-1/2}(\mathring{\mathrm{A}}^T\mathring{\mathrm{A}}+\lambda\mathrm{I})(\mathrm{A}^T\mathrm{A}+\lambda\mathrm{I})^{-1/2}-\mathrm{I}\|_2\le\epsilon$$

and so

$$(1-\epsilon)(\mathrm{A}^T\mathrm{A}+\lambda\mathrm{I})\preceq\mathring{\mathrm{A}}^T\mathring{\mathrm{A}}+\lambda\mathrm{I}\preceq(1+\epsilon)(\mathrm{A}^T\mathrm{A}+\lambda\mathrm{I}).$$

Subtracting $\lambda\mathrm{I}=(\delta/\epsilon)\mathrm{I}$ from all sides, we get

$$(1-\epsilon)\mathrm{A}^T\mathrm{A}-\delta\mathrm{I}\preceq\mathring{\mathrm{A}}^T\mathring{\mathrm{A}}\preceq(1+\epsilon)\mathrm{A}^T\mathrm{A}+\delta\mathrm{I}.$$

Finally, note that, since we set $\mathrm{X}_i=0$ if $\|\mathrm{Y}_{i-1}\|_2\ge\epsilon$, $\|\mathrm{Y}_n\|_2<\epsilon$ implies $\|\mathrm{Y}_i\|_2<\epsilon$ for all $i<n$. We thus have the desired bound on $\tilde{l}_i$ by equation (3.1). □

If we set $c$ in Online-Sample to be proportional to $\log n$ rather than $\log d$, we would be able to take a union bound over all the rows and guarantee that with high probability all the approximate online leverage scores $\tilde{l}_i$ are close to true online leverage scores $l_i$. Thus Theorem 2.2 would imply that Online-Sample only selects $\mathcal{O}(d\log n\log(\|\mathrm{A}\|_2^2/\lambda)/\epsilon^2)$ rows with high probability.

In order to remove the dependency on $n$, we have to sacrifice achieving close approximations to $l_i$ at every step. Instead, we show that the sum of the computed approximate online leverage scores is still small with high probability, using a custom Chernoff bound.

Lemma 3.4. After running Online-Sample, it holds with high probability that

$$\sum_{i=1}^{n}\tilde{l}_i=\mathcal{O}(d\log(\|\mathrm{A}\|_2^2/\lambda)).$$

Proof. Define

$$\delta_i := \log\det(\mathring{\mathrm{A}}_i^T\mathring{\mathrm{A}}_i+\lambda\mathrm{I})-\log\det(\mathring{\mathrm{A}}_{i-1}^T\mathring{\mathrm{A}}_{i-1}+\lambda\mathrm{I}).$$

The proof closely follows the idea from the proof of Theorem 2.2. We will aim to show that large values of $\tilde{l}_i$ correlate with large values of $\delta_i$. Then, the sum of $\delta_i$ can be bounded by the logarithm of the ratio of the determinants of $\mathring{\mathrm{A}}^T\mathring{\mathrm{A}}+\lambda\mathrm{I}$ and $\lambda\mathrm{I}$, giving us a bound on the sum of $\tilde{l}_i$. First, we will show that $\mathbb{E}\left[\exp(\tilde{l}_i/8-\delta_i)\mid\mathring{\mathrm{A}}_{i-1},\ldots,\mathring{\mathrm{A}}_0\right]$ is always at most 1. Note that if row $i$ is sampled by Online-Sample,

$\mathbb{A}_i^T\mathbb{A}_i + \lambda \mathrm{I} = \mathbb{A}_{i-1}^T\mathbb{A}_{i-1} + \lambda \mathrm{I} + \mathrm{a}_i\mathrm{a}_i^T/p_i$ By the matrix determinant lemma (Lemma 3.1), we thus have $\mathrm{e}^{\delta_i} = 1 + \mathrm{a}_i^T(\mathbb{A}_{i-1}^T\mathbb{A}_{i-1} + \lambda \mathrm{I})^{-1}\mathrm{a}_i/p$ in this case. Otherwise, if row $i$ is not sampled, $\delta_i = 0$. Thus,

$$\begin{aligned}\mathbb{E}\left[\exp(\tilde{l}_i/8 - \delta_i) \mid \mathbb{A}_{i-1}, \ldots, \mathbb{A}_0\right] &= p_i \cdot \mathrm{e}^{\tilde{l}_i/8}(1 + \mathrm{a}_i^T(\mathbb{A}_{i-1}^T\mathbb{A}_{i-1} + \lambda \mathrm{I})^{-1}\mathrm{a}_i/p_i)^{-1} + (1-p_i)\mathrm{e}^{\tilde{l}_i/8} \\ &\leq p_i \cdot (1 + \tilde{l}_i/4)(1 + \mathrm{a}_i^T(\mathbb{A}_{i-1}^T\mathbb{A}_{i-1} + \lambda \mathrm{I})^{-1}\mathrm{a}_i/p_i)^{-1} + (1-p_i)(1 + \tilde{l}_i/4). \quad (3.3)\end{aligned}$$

If $c\tilde{l}_i < 1$, we have $p_i = c\tilde{l}_i$ and $\tilde{l}_i = (1+\epsilon)\mathrm{a}_i^T(\mathbb{A}_{i-1}^T\mathbb{A}_{i-1} + \lambda \mathrm{I})^{-1}\mathrm{a}_i$, and so,

$$\begin{aligned}\mathbb{E}\left[\exp(\tilde{l}_i/8 - \delta_i) \mid \mathbb{A}_{i-1}, \ldots, \mathbb{A}_0\right] &\leq c\tilde{l}_i \cdot (1 + \tilde{l}_i/4)(1 + 1/((1+\epsilon)c))^{-1} + (1 - c\tilde{l}_i)(1 + \tilde{l}_i/4) \\ &= (1 + \tilde{l}_i/4)(c\tilde{l}_i(1 + 1/((1+\epsilon)c))^{-1} + 1 - c\tilde{l}_i) \\ &= (1 + \tilde{l}_i/4)(1 - \tilde{l}_i/4) \\ &\leq 1.\end{aligned}$$

Otherwise, we have $p_i = 1$ and so, by (3.3),

$$\begin{aligned}\mathbb{E}\left[\exp(\tilde{l}_i/8 - \delta_i) \mid \mathbb{A}_{i-1}, \ldots, \mathbb{A}_0\right] &\leq (1 + \tilde{l}_i/4)(1 + \mathrm{a}_i^T(\mathbb{A}_{i-1}^T\mathbb{A}_{i-1} + \lambda \mathrm{I})^{-1}\mathrm{a}_i)^{-1} \\ &\leq (1 + \tilde{l}_i/4)(1 + \tilde{l}_i)^{-1} \\ &\leq 1.\end{aligned}$$

We will now analyze the expected product of $\exp(\tilde{l}_i/8 - \delta_i)$ over the first $k$ steps, $\mathbb{E}\left[\exp\left(\sum_{i=1}^k \tilde{l}_i/8 - \delta_i\right)\right]$. Since conditioned on the first $k$ steps, $\exp(\tilde{l}_k/8 - \delta_k)$ is independent of $\exp(\tilde{l}_i/8 - \delta_i)$ for all $i < k$, for $k \geq 1$ we have

$$\begin{aligned}\mathbb{E}\left[\exp\left(\sum_{i=1}^{k} \tilde{l}_i/8 - \delta_i\right)\right] &= \mathbb{E}_{\text{first } k-1 \text{ steps}}\left[\exp\left(\sum_{i=1}^{k-1} \tilde{l}_i/8 - \delta_i\right)\mathbb{E}\left[\exp(\tilde{l}_k/8 - \delta_k) \mid \mathbb{A}_{k-1}, \ldots, \mathbb{A}_0\right]\right] \\ &\leq \mathbb{E}\left[\exp\left(\sum_{i=1}^{k-1} \tilde{l}_i/8 - \delta_i\right)\right],\end{aligned}$$

and so by induction on $k$

$$\mathbb{E}\left[\exp\left(\sum_{i=1}^{n} \tilde{l}_i/8 - \delta_i\right)\right] \leq 1.$$

Hence by Markov's inequality

$$P\left[\sum_{i=1}^{n} \tilde{l}_i > 8d + 8\sum_{i=1}^{n} \delta_i\right] \leq \mathrm{e}^{-d}.$$

By Lemma 3.3, with high probability we have $\mathbb{A}^T\tilde{\mathrm{A}} + \lambda \mathrm{I} \preceq (1+\epsilon)(\mathrm{A}^T\mathrm{A} + \lambda \mathrm{I})$. We also have with high probability

$$\begin{aligned}\det(\mathbb{A}^T\tilde{\mathrm{A}} + \lambda \mathrm{I}) &\leq (1+\epsilon)^d(\|\mathrm{A}\|_2^2 + \lambda)^d, \\ \log\det(\mathbb{A}^T\tilde{\mathrm{A}} + \lambda \mathrm{I}) &\leq d(1 + \log(\|\mathrm{A}\|_2^2 + \lambda)).\end{aligned}$$

Hence, with high probability it holds that

$$\begin{aligned}\sum_{i=1}^{n} \delta_i &= \log\det(\mathbb{A}^T\tilde{\mathrm{A}} + \lambda \mathrm{I}) - d\log(\lambda) \\ &\leq d(1 + \log(\|\mathrm{A}\|_2^2 + \lambda) - \log(\lambda)) \\ &= d(1 + \log(1 + \|\mathrm{A}\|_2^2/\lambda)).\end{aligned}$$

And so, with high probability,

$$\begin{aligned}\sum_{i=1}^{n} \tilde{l}_i &\le 8d + 8\sum_{i=1}^{n} \delta_i \\ &\le 16d + 8d\log(1 + \|\mathrm{A}\|_2^2/\lambda) \\ &= \mathcal{O}(d\log(\|\mathrm{A}\|_2^2/\lambda)).\end{aligned}$$

□

Proof of Theorem 2.3. The statement follows immediately from Lemmas 3.3 and 3.4. □

Observe that by Theorem 2.3, Online-Sample stores $O(d\log d\log(\epsilon\|\mathrm{A}\|_2^2/\delta)/\epsilon^2)$ rows in memory. We now consider a simple modification of the algorithm, Slim-Sample (Figure 2), that removes the $1/\epsilon^2$ factor from the working memory usage with no additional cost.

$\mathbb{A} =$ Slim-Sample$(\mathrm{A}, \epsilon, \delta)$, where A is an $n \times d$ matrix with rows $\mathrm{a}_1, \ldots, \mathrm{a}_n$,
$\epsilon \in (0,1)$, $\delta > 0$.

1. Set $\lambda := \delta/\epsilon$, $c := 8\log d/\epsilon^2$.
2. Let $\mathbb{A}_0$ be a $0 \times d$ matrix.
3. Let $\tilde{l}_1, \ldots, \tilde{l}_n$ be the approximate online leverage scores computed by an independent instance of Online-Sample$(\mathrm{A}, 1/2, \delta/(2\epsilon))$.
4. For $i = 1, \ldots, n$:
   (a) Let $p_i := \min(c\tilde{l}_i, 1)$.
   (b) Set $\mathbb{A}_i := \begin{cases} \begin{bmatrix} \mathbb{A}_{i-1} \\ \mathrm{a}_i/\sqrt{p_i} \end{bmatrix} & \text{with probability } p_i, \\ \mathbb{A}_{i-1} & \text{otherwise.} \end{cases}$
5. Return $\mathbb{A} := \mathbb{A}_n$.

Figure 2: The low-memory online sampling algorithm

Lemma 3.5. Let $\mathbb{A}$ be the matrix returned by Slim-Sample$(\mathrm{A}, \epsilon, \delta)$. Then, with high probability,

$$(1-\epsilon)\mathrm{A}^T\mathrm{A} - \delta\mathrm{I} \preceq \mathbb{A}^T\mathbb{A} \preceq (1+\epsilon)\mathrm{A}^T\mathrm{A} + \delta\mathrm{I},$$

and the number of rows in $\mathbb{A}$ is $\mathcal{O}(d\log d\log(\epsilon\|\mathrm{A}\|_2^2/\delta)/\epsilon^2)$.

Moreover, with high probability, the memory requirement of Slim-Sample is dominated by storing $\mathcal{O}(d\log d\log(\epsilon\|\mathrm{A}\|_2^2/\delta))$ rows of A.

Proof. As the samples are independent, the statement follows from Theorem 2.1 and Lemmas 3.3 and 3.4. □

# 4 Asymptotically optimal algorithm

In addition to sampling by online leverage scores, we introduce a row sampling algorithm, Online-BSS (Figure 3), which improves the row count of Online-Sample by a $\log d$ factor, to

$$\mathcal{O}(d\log(\epsilon\|\mathrm{A}\|_2^2/\delta)/\epsilon^2).$$

This improved bound matches the lower bound for online sampling given in Theorem 5.1. This approach uses a variant of the deterministic "BSS" method, introduced by Batson, Spielman, and Srivastava in [2]. It is well known that this method yields spectral approximations with a $\log d$ factor fewer rows than

leverage scores sampling in the offline setting, and we show that this improvement extends to online approximation.

Unlike the original BSS algorithm of [2], our algorithm is randomized. It is similar to, and inspired by, the randomized version of BSS from [19], especially "Algorithm 1" from that paper. In both algorithms, like in online leverage score sampling, when a new row is processed, a probability $p_i$ is assigned to it, and it is kept with probability $p_i$ and rejected otherwise. The key difference between the algorithms is in the definition of $p_i$. Like Online-Sample, at each step, Online-BSS maintains a row sample $\mathbb{A}_i$ which approximates the matrix $\mathrm{A}_i$ that has been seen so far. However, $p_i$ cannot be computed solely based on $\mathbb{A}_{i-1}$—it is necessary to "remember" the entire input. Thus, Online-BSS is not memory efficient, using $O(d^2)$ space. One may improve the memory dependence by simply running Online-BSS on the output stream of rows produced by Online-Sample. This reduces the storage cost to the size of that output spectral approximation. Of course, this does not mean that Online-BSS leads to a space savings over Online-Sample. However the number of rows in its output stream will be less than that of Online-Sample, by a $\log d$ factor.

We also remark that Online-Sample gives bounds on both the size of the output spectral approximation and its accuracy with high probability. In contrast, Online-BSS gives an expected bound on the output size, while it never fails to output a correct spectral approximation. These guarantees are similar to those given in [19]. Below, we give present the performance guarantees of Online-BSS and its analysis.

Theorem 4.1. Let $\mathbb{A}$ be the matrix output by Online-BSS$(\mathrm{A}, \epsilon, \delta)$ (Figure 3).

1. We always have $(1-\epsilon)\mathrm{A}^T\mathrm{A} - \delta\mathrm{I} \prec \mathbb{A}^T\mathbb{A} \prec (1+\epsilon)\mathrm{A}^T\mathrm{A} + \delta\mathrm{I}$.
2. The expected number of rows in $\mathbb{A}$ is $\mathcal{O}(d\log(\epsilon\|\mathrm{A}\|_2^2/\delta)/\epsilon^2)$.

$\mathbb{A}$ = Online-BSS$(\mathrm{A}, \epsilon, \delta)$, where A is an $n \times d$ matrix with rows $\mathrm{a}_1, \ldots, \mathrm{a}_n$,
$\epsilon \in (0,1)$, $\delta > 0$.

1. Set $c_U = \frac{2}{\epsilon} + 1$ and $c_L = \frac{2}{\epsilon} - 1$.
2. Let $\mathbb{A}_0$ be a $0 \times d$ matrix, $\mathrm{B}_0^U = \delta\mathrm{I}$, $\mathrm{B}_0^L = -\delta\mathrm{I}$.
3. For $i = 1, \ldots, n$:
   (a) Let $\mathrm{X}_{i-1}^U = (\mathrm{B}_{i-1}^U - \mathbb{A}_{i-1}^T\mathbb{A}_{i-1})$, $\mathrm{X}_{i-1}^L = (\mathbb{A}_{i-1}^T\mathbb{A}_{i-1} - \mathrm{B}_{i-1}^L)$.
   (b) Let $p_i := \min(c_U \mathrm{a}_i^T(\mathrm{X}_{i-1}^U)^{-1}\mathrm{a}_i + c_L \mathrm{a}_i^T(\mathrm{X}_{i-1}^L)^{-1}\mathrm{a}_i\,, 1)$.
   (c) Set $\mathbb{A}_i := \begin{cases} \begin{bmatrix} \mathbb{A}_{i-1} \\ \mathrm{a}_i/\sqrt{p_i} \end{bmatrix} & \text{with probability } p_i, \\ \mathbb{A}_{i-1} & \text{otherwise.} \end{cases}$
   (d) Set $\mathrm{B}_i^U = \mathrm{B}_{i-1}^U + (1+\epsilon)\mathrm{a}_i\mathrm{a}_i^T$, $\mathrm{B}_i^L = \mathrm{B}_{i-1}^L + (1-\epsilon)\mathrm{a}_i\mathrm{a}_i^T$.
4. Return $\mathbb{A} := \mathbb{A}_n$.

Figure 3: The Online BSS Algorithm

Proof of Theorem 4.1 Part 1. As in [2], a key of idea of Online-BSS is to maintain two matrices, $\mathrm{B}_i^U$ and $\mathrm{B}_i^L$, acting as upper and lower "barriers." We will prove that the current approximation $\mathbb{A}_i$ always falls between them:

$$\mathrm{B}_i^L \prec \mathbb{A}_i^T\mathbb{A}_i \prec \mathrm{B}_i^U\,. \tag{4.1}$$

Equivalently, $\mathrm{X}_i^U$ and $\mathrm{X}_i^L$ will always remain positive definite. Since, at the completion of the algorithm, $\mathrm{B}_n^U = (1+\epsilon)\mathrm{A}^T\mathrm{A} + \delta\mathrm{I}$ and $\mathrm{B}_n^L = (1-\epsilon)\mathrm{A}^T\mathrm{A} - \delta\mathrm{I}$ this ensures that the final approximation always

satisfies the approximation bound in claim (1) of the theorem. $p_i$ is chosen at step 3(b) to ensure this invariant—if either $\mathrm{X}_i^U$ or $\mathrm{X}_i^L$ are too small (we are too close to one of the barriers) then at least one of $\mathrm{a}_i^T(\mathrm{X}_{i-1}^U)^{-1}\mathrm{a}_i$ or $\mathrm{a}_i^T(\mathrm{X}_{i-1}^L)^{-1}\mathrm{a}_i$ will be large and so $p_i$ will be large.

We can prove this invariant if (4.1) holds, by induction on $i$. The base case follows from the initialization of $\mathbb{A}_0$ with $\mathbb{A}_0^T\mathbb{A}_0 = 0$, $\mathrm{B}_0^U = \delta\mathrm{I}$, and $\mathrm{B}_0^L = -\delta\mathrm{I}$ since clearly $-\delta\mathrm{I} \prec 0 \prec \delta\mathrm{I}$. For each successive step, we consider two possibilities.

Case 1: $p_i = 1$. When $p_i = 1$, $\mathbb{A}_i^T\mathbb{A}_i = \mathbb{A}_{i-1}^T\mathbb{A}_{i-1} + \mathrm{a}_i\mathrm{a}_i^T$. Since we set $\mathrm{B}_i^U = \mathrm{B}_{i-1}^U + (1+\epsilon)\mathrm{a}_i\mathrm{a}_i^T$ and $\mathrm{B}_i^L = \mathrm{B}_{i-1}^L + (1-\epsilon)\mathrm{a}_i\mathrm{a}_i^T$, we can see that $\mathrm{X}_i^U = \mathrm{X}_{i-1}^U + \epsilon\mathrm{a}_i\mathrm{a}_i^T$ and $\mathrm{X}_i^L = \mathrm{X}_{i-1}^L + \epsilon\mathrm{a}_i\mathrm{a}_i^T$. Since by the induction assumption, $\mathrm{X}_{i-1}^U$ and $\mathrm{X}_{i-1}^L$ are both positive definite, so are $\mathrm{X}_i^U$ and $\mathrm{X}_i^L$, giving the claim.

Case 2: $p_i < 1$. In this case, with probability $p_i$, $\mathbb{A}_i^T\mathbb{A}_i = \mathbb{A}_{i-1}^T\mathbb{A}_{i-1} + \mathrm{a}_i\mathrm{a}_i^T/p$. With probability $1-p_i$, $\mathbb{A}_i^T\mathbb{A}_i = \mathbb{A}_{i-1}^T\mathbb{A}_{i-1}$. Thus, in any case, $\mathbb{A}_i^T\mathbb{A}_i \prec \mathbb{A}_{i-1}^T\mathbb{A}_{i-1} + \mathrm{a}_i\mathrm{a}_i^T/p$. In turn, $\mathrm{X}_i^U \succeq \mathrm{X}_{i-1}^U - \mathrm{a}_i\mathrm{a}_i^T/p_i$. Observe that $c_U = 2/\epsilon + 1 > 1$ and thus $p_i > \mathrm{a}_i^T(\mathrm{X}_{i-1}^U)^{-1}\mathrm{a}_i$. This gives $\mathrm{X}_{i-1}^U \succ \mathrm{a}_i\mathrm{a}_i^T/p_i$ and so since $\mathrm{X}_i^U \succeq \mathrm{X}_{i-1}^U - \mathrm{a}_i\mathrm{a}_i^T/p_i$, it must be postive definite.

Analogously, since $\mathrm{B}_i^L = \mathrm{B}_{i-1}^L + (1-\epsilon)\mathrm{a}_i\mathrm{a}_i^T$, we have $\mathrm{X}_i^L \succeq \mathrm{X}_{i-1}^L - (1-\epsilon)\mathrm{a}_i\mathrm{a}_i^T \succeq \mathrm{X}_{i-1}^L - \mathrm{a}_i\mathrm{a}_i^T$. Since $c_L = 2/\epsilon - 1 > 1$ for $\epsilon \in (0,1)$, and since $p_i < 1$ (by the fact that we are in Case 2), we have $\mathrm{a}_i^T(\mathrm{X}_{i-1}^L)^{-1}\mathrm{a}_i < 1$. This in turn gives $\mathrm{X}_{i-1}^L \succ \mathrm{a}_i\mathrm{a}_i^T$ and thus since $\mathrm{X}_i^L \succeq \mathrm{X}_{i-1}^L - \mathrm{a}_i\mathrm{a}_i^T$, it must be positive definite, giving the claim in this case.

Thus, we have shown (4.1) for all $i$. In particular, $\mathrm{B}_n^U \prec \mathbb{A}^T\mathbb{A} \prec \mathrm{B}_n^L$. We can see by construction that

$$\mathrm{B}_n^U = (1+\epsilon)\mathrm{A}^T\mathrm{A} + \delta\mathrm{I} \text{ and } \mathrm{B}_n^L = (1-\epsilon)\mathrm{A}^T\mathrm{A} - \delta\mathrm{I}.$$

Thus, we have $(1-\epsilon)\mathrm{A}^T\mathrm{A} - \delta\mathrm{I} \prec \mathbb{A}^T\mathbb{A} \prec (1+\epsilon)\mathrm{A}^T\mathrm{A} + \delta\mathrm{I}$, which gives the first claim of the theorem. □

In our proof of the second claim, bounding the expected number of rows sampled, we will need the following technical lemma, which is derived from the Sherman–Morrison formula [24].

Lemma 4.2. Given a positive definite matrix X, two vectors u and v, two scalar multipliers $a$ and $b$, and a probability $p$, define the random variable $\widehat{\mathrm{X}}$ to be $\mathrm{X} - a\mathrm{u}\mathrm{u}^T$ with probability $p$ and $\mathrm{X} - b\mathrm{u}\mathrm{u}^T$ otherwise. Then if $\mathrm{u}^T\mathrm{X}^{-1}\mathrm{u} = 1$,

$$\mathbb{E}\left[\mathrm{v}^T\widehat{\mathrm{X}}^{-1}\mathrm{v} - \mathrm{v}^T\mathrm{X}^{-1}\mathrm{v}\right] = (\mathrm{v}^T\mathrm{X}^{-1}\mathrm{u})^2 \cdot \frac{pa + (1-p)b - ab}{(1-a)(1-b)}.$$

Proof. We apply the Sherman–Morrison formula to each of the two possibilities ($\widehat{\mathrm{X}} = \mathrm{X}\mathrm{X} - a\mathrm{u}\mathrm{u}^T$ and $\widehat{\mathrm{X}} = \mathrm{X} + b\mathrm{u}\mathrm{u}^T$ respectively). These give respective $\widehat{\mathrm{X}}^{-1}$ values of

$$\widehat{\mathrm{X}}^{-1} = \mathrm{X}^{-1} + a \cdot \frac{\mathrm{X}^{-1}\mathrm{u}\mathrm{u}^T\mathrm{X}^{-1}}{1 - a\mathrm{u}^T\mathrm{X}^{-1}\mathrm{u}} = \mathrm{X}^{-1} + \frac{a}{1-a} \cdot \mathrm{X}^{-1}\mathrm{u}\mathrm{u}^T\mathrm{X}^{-1}$$

and

$$\widehat{\mathrm{X}}^{-1} = \mathrm{X}^{-1} + b \cdot \frac{\mathrm{X}^{-1}\mathrm{u}\mathrm{u}^T\mathrm{X}^{-1}}{1 - b\mathrm{u}^T\mathrm{X}^{-1}\mathrm{u}} = \mathrm{X}^{-1} + \frac{b}{1-b} \cdot \mathrm{X}^{-1}\mathrm{u}\mathrm{u}^T\mathrm{X}^{-1}.$$

The values of $\mathrm{v}^T\widehat{\mathrm{X}}^{-1}\mathrm{v} - \mathrm{v}^T\mathrm{X}^{-1}\mathrm{v}$ are then respectively

$$\frac{a}{1-a} \cdot \mathrm{v}^T\mathrm{X}^{-1}\mathrm{u}\mathrm{u}^T\mathrm{X}^{-1}\mathrm{v} = (\mathrm{v}^T\mathrm{X}^{-1}\mathrm{u})^2 \cdot \frac{a}{1-a}$$

and

$$\frac{b}{1-b} \cdot \mathrm{v}^T\mathrm{X}^{-1}\mathrm{u}\mathrm{u}^T\mathrm{X}^{-1}\mathrm{v} = (\mathrm{v}^T\mathrm{X}^{-1}\mathrm{u})^2 \cdot \frac{b}{1-b}.$$

Combining these gives the stated result. □

Proof of Theorem 4.1 Part 2. We will show that the probability that row $\mathrm{a}_i$ is included in $\mathbb{A}$ is at most $8/\epsilon^2 \cdot l_i$, where $l_i$ is the online $2\delta/\epsilon$-ridge leverage score of $\mathrm{a}_i$, i. e.,

$$l_i = \min(\mathrm{a}_i^T\left(\mathrm{A}_{i-1}^T\mathrm{A}_{i-1} + 2\delta/\epsilon \cdot \mathrm{I}\right)^{-1}\mathrm{a}_i\,, 1).$$

Since $\sum_{i=1}^{n} l_i = \mathcal{O}(d\log(\epsilon\|\mathrm{A}\|_2^2/\delta))$ by Theorem 2.2, this implies that $\mathbb{A}$ has $\mathcal{O}(d\log(\epsilon\|\mathrm{A}\|_2^2/\delta)/\epsilon^2)$ rows in expectation, completing the second claim of the theorem.

First, we introduce some notation to help in the analysis. Let $q_i$ be the probability that row $\mathrm{a}_i$ is sampled in the algorithm. Note that $q_i$ is fixed and we seek to prove that $q_i \leq 8/\epsilon^2 \cdot l_i$. The probability $p_i$ that $\mathrm{a}_i$ is sampled at step $i$ is a random variable. We have $q_i = \mathbb{E}[p_i]$. Thus it suffices to prove $\mathbb{E}[p_i] \leq 8/\epsilon^2 \cdot l_i$. We define

$$\mathrm{C}_{i,j}^U = \delta\mathrm{I} + \frac{\epsilon}{2}\mathrm{A}_i^T\mathrm{A}_i + \left(1+\frac{\epsilon}{2}\right)\mathrm{A}_j^T\mathrm{A}_j$$
$$\mathrm{C}_{i,j}^L = -\delta\mathrm{I} - \frac{\epsilon}{2}\mathrm{A}_i^T\mathrm{A}_i + \left(1-\frac{\epsilon}{2}\right)\mathrm{A}_j^T\mathrm{A}_j\,.$$

Note that $\mathrm{C}_{i,i}^U = \mathrm{B}_i^U$, $\mathrm{C}_{i,i}^L = \mathrm{B}_i^L$, and for $j \leq i$, $\mathrm{C}_{i,j}^U \succeq \mathrm{B}_j^U$ and $\mathrm{C}_{i,j}^L \preceq \mathrm{B}_j^L$. We can then define

$$\mathrm{Y}_{i,j}^U = \mathrm{C}_{i,j}^U - \mathbb{A}_j^T\mathbb{A}_j$$
$$\mathrm{Y}_{i,j}^L = \mathbb{A}_j^T\mathbb{A}_j - \mathrm{C}_{i,j}^L\,.$$

We then have, similarly, $\mathrm{Y}_{i,i}^U = \mathrm{X}_i^U$, $\mathrm{Y}_{i,i}^L = \mathrm{X}_i^L$, and for $j \leq i$, $\mathrm{Y}_{i,j}^U \succeq \mathrm{X}_j^U$ and $\mathrm{Y}_{i,j}^L \succeq \mathrm{X}_j^L$.

Assume that $l_i < 1$. Otherwise, since $p_i \leq 1$ (it is a probability), we trivially have $\mathbb{E}[p_i] \leq 8/\epsilon^2 \cdot l_i$ as desired. Now, note that for all $i > 0$,

$$\begin{aligned}\mathrm{a}_i^T(\mathrm{Y}_{i,0}^U)^{-1}\mathrm{a}_i &= \mathrm{a}_i^T(\mathrm{Y}_{i,0}^L)^{-1}\mathrm{a}_i \\ &= \mathrm{a}_i^T\left(\frac{\epsilon}{2}\mathrm{A}_i^T\mathrm{A}_i + \delta\mathrm{I}\right)^{-1}\mathrm{a}_i \\ &= \frac{2}{\epsilon}\mathrm{a}_i^T\left(\mathrm{A}_i^T\mathrm{A}_i + \frac{2\delta}{\epsilon}\mathrm{I}\right)^{-1}\mathrm{a}_i \quad \leq \quad \frac{2}{\epsilon}l_i\,. \end{aligned} \tag{4.2}$$

Next, we will show that for $j < i-1$,

$$\mathbb{E}\left[\mathrm{a}_i^T(\mathrm{Y}_{i-1,j+1}^U)^{-1}\mathrm{a}_i\right] \leq \mathbb{E}\left[\mathrm{a}_i^T(\mathrm{Y}_{i-1,j}^U)^{-1}\mathrm{a}_i\right] \tag{4.3}$$

and

$$\mathbb{E}\left[\mathrm{a}_i^T(\mathrm{Y}_{i-1,j+1}^L)^{-1}\mathrm{a}_i\right] \leq \mathbb{E}\left[\mathrm{a}_i^T(\mathrm{Y}_{i-1,j}^L)^{-1}\mathrm{a}_i\right]\,. \tag{4.4}$$

Combined with (4.2) and the fact that $\mathrm{Y}_{i,i}^U = \mathrm{X}_i^U$, $\mathrm{Y}_{i,i}^L = \mathrm{X}_i^L$, (4.3) and (4.4) give

$$\begin{aligned}\mathbb{E}[p_i] &= c_U\mathbb{E}\left[\mathrm{a}_i^T(\mathrm{X}_{i-1}^U)^{-1}\mathrm{a}_i\right] + c_L\mathbb{E}\left[\mathrm{a}_i^T(\mathrm{X}_{i-1}^L)^{-1}\mathrm{a}_i\right] \\ &= c_U\mathbb{E}\left[\mathrm{a}_i^T(\mathrm{Y}_{i-1,i-1}^U)^{-1}\mathrm{a}_i\right] + c_L\mathbb{E}\left[\mathrm{a}_i^T(\mathrm{Y}_{i-1,i-1}^L)^{-1}\mathrm{a}_i\right] \\ &\leq c_U\mathbb{E}\left[\mathrm{a}_i^T(\mathrm{Y}_{i-1,0}^U)^{-1}\mathrm{a}_i\right] + c_L\mathbb{E}\left[\mathrm{a}_i^T(\mathrm{Y}_{i-1,0}^L)^{-1}\mathrm{a}_i\right] \\ &= \frac{2}{\epsilon}l_i\cdot(c_U+c_L) = \frac{8}{\epsilon^2}l_i\,.\end{aligned}$$

The last equality follows from the fact that in Online-BSS we set $c_U = 2/\epsilon + 1$ and $c_L = 2/\epsilon - 1$. This completes the claim that for all $i$, the probability $q_i$ that row $\mathrm{a}_i$ is sampled is bounded by $q_i = \mathbb{E}[p_i] \leq 8/\epsilon^2 \cdot l_i$, giving the second part of Theorem 4.1.

It remains to prove (4.3) and (4.4). To do this we will show a somewhat stronger statement: conditioned on any choices for the first $j$ rows, the expected value of $\mathrm{a}_i^T(\mathrm{Y}_{i-1,j+1}^U)^{-1}\mathrm{a}_i$ is no larger than that of $\mathrm{a}_i^T(\mathrm{Y}_{i-1,j}^U)^{-1}\mathrm{a}_i$, and analogously for $(\mathrm{Y}_{i-1,j+1}^L)^{-1}$. Similar to the proof of part 1, we consider two cases: Case 1: $p_{j+1} = 1$. In that case, the positive semidefinite matrix $\mathrm{a}_{j+1}\mathrm{a}_{j+1}^T$ is added at step $j+1$ to give $\mathbb{A}_{j+1}^T\mathbb{A}_{j+1} = \mathbb{A}_j^T\mathbb{A}_j + \mathrm{a}_{j+1}\mathrm{a}_{j+1}^T$. This gives that

$$\begin{aligned}\mathrm{Y}_{i-1,j+1}^U &= \mathrm{C}_{i,j+1}^U - \mathbb{A}_{j+1}^T\mathbb{A}_{j+1} \\ &= \mathrm{C}_{i,j}^U + \left(1+\frac{\epsilon}{2}\right)\mathrm{a}_{j+1}\mathrm{a}_{j+1}^T - (\mathbb{A}_j^T\mathbb{A}_j + \mathrm{a}_{j+1}\mathrm{a}_{j+1}^T) \\ &= \mathrm{Y}_{i-1,j}^U + \frac{\epsilon}{2}\cdot\mathrm{a}_{j+1}\mathrm{a}_{j+1}^T.\end{aligned}$$

Thus we have $\mathrm{Y}^U_{i-1,j+1} \succeq \mathrm{Y}^U_{i-1,j}$ and so $\mathrm{a}_i^T(\mathrm{Y}^U_{i-1,j+1})^{-1}\mathrm{a}_i \leq \mathrm{a}_i^T(\mathrm{Y}^U_{i-1,j})^{-1}\mathrm{a}_i$, giving (4.3). An analogous argument holds for $\mathrm{Y}^L_{i-1,j+1}$, giving (4.4).

Case 2: $p_{j+1} < 1$. This case is more tricky. Importantly, by how $p_{j+1}$ is set in step 3(b) of Online-BSS and by the observation that $\mathrm{Y}^U_{i-1,j} \succeq \mathrm{X}^U_j$ and $\mathrm{Y}^L_{i-1,j} \succeq \mathrm{X}^L_j$ for $j \leq i-1$ (recall that we must prove (4.3) and (4.4) under the assumption that $j \leq i-1$), we have

$$p_{j+1} \geq c_U \cdot \mathrm{a}_{j+1}^T(\mathrm{X}^U_j)^{-1}\mathrm{a}_{j+1} \geq c_U \cdot \mathrm{a}_{j+1}^T(\mathrm{Y}^U_{i-1,j})^{-1}\mathrm{a}_{j+1} \tag{4.5}$$

and

$$p_{j+1} \geq c_L \cdot \mathrm{a}_{j+1}^T(\mathrm{X}^L_j)^{-1}\mathrm{a}_{j+1} \geq c_L \cdot \mathrm{a}_{j+1}^T(\mathrm{Y}^L_{i-1,j})^{-1}\mathrm{a}_{j+1}. \tag{4.6}$$

Now, we define $\mathrm{w}_{j+1} = \mathrm{a}_{j+1}/\sqrt{p_{j+1}}$ and additionally

$$\begin{aligned}
s^U_{j+1} &= \mathrm{w}_{j+1}^T(\mathrm{Y}^U_{i-1,j})^{-1}\mathrm{w}_{j+1}^T \\
s^L_{j+1} &= \mathrm{w}_{j+1}^T(\mathrm{Y}^L_{i-1,j})^{-1}\mathrm{w}_{j+1}^T \\
\mathrm{u}^U_{j+1} &= \frac{\mathrm{w}_{j+1}}{\sqrt{s^U_{j+1}}} \\
\mathrm{u}^L_{j+1} &= \frac{\mathrm{w}_{j+1}}{\sqrt{s^L_{j+1}}}.
\end{aligned}$$

We then deploy Lemma 4.2 to bound the expectations in (4.3) and (4.4).

Upper barrier bound (4.3): Here we apply Lemma 4.2 with $\mathrm{X} = \mathrm{Y}^U_{i-1,j}$, $\mathrm{u} = \mathrm{u}^U_{j+1}$, $\mathrm{v} = \mathrm{a}_i^T$, $a = s^U_{j+1}(1 - p_{j+1}(1+\epsilon/2))$, $b = -s^U_{j+1}p_{j+1}(1+\epsilon/2)$, $p = p_{j+1}$. Note that we have

$$\mathrm{u}^T\mathrm{X}^{-1}\mathrm{u} = 1/s^U_{j+1} \cdot \mathrm{w}_{j+1}^T(\mathrm{Y}^U_{i-1,j})^{-1}\mathrm{w}_{j+1} = 1$$

as required. Additionally, with probability $p_{j+1}$,

$$\begin{aligned}
\widehat{\mathrm{X}} &= \mathrm{X} - a\mathrm{u}\mathrm{u}^T \\
&= \mathrm{Y}^U_{i-1,j} - s^U_{j+1}(1 - p_{j+1}(1+\epsilon/2)) \cdot \mathrm{u}^U_{j+1}{\mathrm{u}^U_{j+1}}^T \\
&= \mathrm{Y}^U_{i-1,j} - (1 - p_{j+1}(1+\epsilon/2)) \cdot \mathrm{w}_{j+1}{\mathrm{w}_{j+1}}^T \\
&= \mathrm{Y}^U_{i-1,j} - \frac{1}{p_{j+1}} \cdot \mathrm{a}_{j+1}{\mathrm{a}_{j+1}}^T + (1+\epsilon/2)\mathrm{a}_{j+1}{\mathrm{a}_{j+1}}^T.
\end{aligned}$$

Similarly, with probability $1 - p_{j+1}$,

$$\begin{aligned}
\widehat{\mathrm{X}} &= \mathrm{X} - b\mathrm{u}\mathrm{u}^T \\
&= \mathrm{Y}^U_{i-1,j} + s^U_{j+1}p_{j+1}(1+\epsilon/2)\mathrm{u}^U_{j+1}{\mathrm{u}^U_{j+1}}^T \\
&= \mathrm{Y}^U_{i-1,j} + (1+\epsilon/2)\mathrm{a}_{j+1}{\mathrm{a}_{j+1}}^T.
\end{aligned}$$

That is, $\widehat{\mathrm{X}} = \mathrm{Y}^U_{i-1,j+1}$. Thus, Lemma 4.2 gives

$$\begin{aligned}
\mathbb{E}\left[\mathrm{a}_i^T\widehat{\mathrm{X}}^{-1}\mathrm{a}_i - \mathrm{v}^T\mathrm{X}^{-1}\mathrm{v}\right] &= \mathbb{E}\left[\mathrm{a}_i^T{\mathrm{Y}^U_{i-1,j+1}}^{-1}\mathrm{a}_i\right] - \mathbb{E}\left[\mathrm{a}_i^T{\mathrm{Y}^U_{i-1,j}}^{-1}\mathrm{a}_i\right] \\
&= (\mathrm{v}^T\mathrm{X}^{-1}\mathrm{u})^2 \cdot \frac{pa + (1-p)b - ab}{(1-a)(1-b)}.
\end{aligned}$$

To prove (4.3) it suffices to show that

$$\frac{pa + (1-p)b - ab}{(1-a)(1-b)}$$

is non-positive. Letting $r = \mathrm{a}_{j+1}^T(\mathrm{Y}^U_{i-1,j})^{-1}\mathrm{a}_{j+1}^T$ we can write $a = r \cdot (1/p_{j+1} - (1+\epsilon/2))$ and $b = -r \cdot (1+\epsilon/2) < 0$. By (4.5), $r \leq p_{j+1}/c_U$ and thus $a \leq r/p_{j+1} \leq 1/c_U = 1/(2/\epsilon + 1) < 1$. Thus, the

denominator $(1-a)(1-b)$ is positive, and so it remains to show that the numerator $pa+(1-p)b-ab$ is non-positive. We can write

$$\begin{aligned} pa+(1-p)b-ab &= -r\cdot\frac{\epsilon}{2}+r^2\cdot\left(\frac{1}{p_{j+1}}-\left(1+\frac{\epsilon}{2}\right)\right)\cdot\left(1+\frac{\epsilon}{2}\right)\\ &\le -r\cdot\frac{\epsilon}{2}+r^2\cdot\left(\frac{1}{p_{j+1}}\right)\cdot\left(1+\frac{\epsilon}{2}\right)\\ &\le -r\cdot\frac{\epsilon}{2}+r\cdot\frac{1}{2/\epsilon+1}\cdot\left(1+\frac{\epsilon}{2}\right)\\ &= -r\cdot\frac{\epsilon}{2}+r\cdot\frac{\epsilon}{2}\le 0. \end{aligned}$$

This completes the argument that $\mathbb{E}\left[\mathrm{a}_i^T{\mathrm{Y}^U_{i-1,j+1}}^{-1}\mathrm{a}_i\right]-\mathbb{E}\left[\mathrm{a}_i^T{\mathrm{Y}^U_{i-1,j}}^{-1}\mathrm{a}_i\right]\le 0$, giving (4.3).

Lower barrier bound (4.4): For the lower barrier bound we give a similar argument. We use $\mathrm{X}=\mathrm{Y}^L_{i-1,j}$, $\mathrm{u}=\mathrm{u}^L_{j+1}$, $\mathrm{v}=\mathrm{a}_i^T$, $a=-s^L_{j+1}(1-p_{j+1}(1-\epsilon/2))$, $b=s^L_{j+1}p_{j+1}(1-\epsilon/2)$, and $p=p_{j+1}$. We again have

$$\mathrm{u}^T\mathrm{X}^{-1}\mathrm{u}=1/s^L_{j+1}\cdot\mathrm{w}^T_{j+1}(\mathrm{Y}^L_{i-1,j})^{-1}\mathrm{w}_{j+1}=1,$$

as required. Additionally, with probability $p_{j+1}$, we have

$$\begin{aligned} \widehat{\mathrm{X}} &= \mathrm{X}-a\mathrm{u}\mathrm{u}^T\\ &= \mathrm{Y}^L_{i-1,j}+s^L_{j+1}(1-p_{j+1}(1-\epsilon/2))\cdot\mathrm{u}^L_{j+1}{\mathrm{u}^L_{j+1}}^T\\ &= \mathrm{Y}^L_{i-1,j}+\frac{1}{p_{j+1}}\cdot\mathrm{a}_{j+1}{\mathrm{a}_{j+1}}^T-(1-\epsilon/2)\mathrm{a}_{j+1}{\mathrm{a}_{j+1}}^T. \end{aligned}$$

Similarly, with probability $1-p_{j+1}$,

$$\begin{aligned} \widehat{\mathrm{X}} &= \mathrm{X}-b\mathrm{u}\mathrm{u}^T\\ &= \mathrm{Y}^L_{i-1,j}-s^L_{j+1}p_{j+1}(1-\epsilon/2)\mathrm{u}^L_{j+1}{\mathrm{u}^L_{j+1}}^T\\ &= \mathrm{Y}^L_{i-1,j}-(1-\epsilon/2)\mathrm{a}_{j+1}{\mathrm{a}_{j+1}}^T. \end{aligned}$$

That is, $\widehat{\mathrm{X}}=\mathrm{Y}^L_{i-1,j+1}$. Thus, Lemma 4.2 gives

$$\begin{aligned} \mathbb{E}\left[\mathrm{a}_i^T\widehat{\mathrm{X}}^{-1}\mathrm{a}_i-\mathrm{v}^T\mathrm{X}^{-1}\mathrm{v}\right] &= \mathbb{E}\left[\mathrm{a}_i^T(\mathrm{Y}^L_{i-1,j+1})^{-1}\mathrm{a}_i\right]-\mathbb{E}\left[\mathrm{a}_i^T(\mathrm{Y}^L_{i-1,j})^{-1}\mathrm{a}_i\right]\\ &= (\mathrm{v}^T\mathrm{X}^{-1}\mathrm{u})^2\cdot\frac{pa+(1-p)b-ab}{(1-a)(1-b)}. \end{aligned}$$

Again, to prove (4.4) it suffices to show that

$$\frac{pa+(1-p)b-ab}{(1-a)(1-b)}$$

is non-positive. Let $r=\mathrm{a}^T_{j+1}(\mathrm{Y}^L_{i-1,j})^{-1}\mathrm{a}_{j+1}$. We can write $a=-r\left(1/p_{j+1}-(1-\epsilon/2)\right)<0$ and $b=r\left(1-\epsilon/2\right)$. Note that by (4.6), $r\le p_{j+1}/c_L=p_{j+1}/(2/\epsilon-1)<1$, and thus $b<1$. So the denominator $(1-a)(1-b)$ is positive. It thus remains to show that the numerator $pa+(1-p)b-ab$ is non-positive. We simplify this numerator as

$$\begin{aligned} pa+(1-p)b-ab &= -r\cdot\frac{\epsilon}{2}+r^2\left(\frac{1}{p_{j+1}}-\left(1-\frac{\epsilon}{2}\right)\right)\cdot\left(1-\frac{\epsilon}{2}\right)\\ &\le -r\cdot\frac{\epsilon}{2}+r^2\cdot\left(\frac{1}{p_{j+1}}\right)\cdot\left(1-\frac{\epsilon}{2}\right)\\ &\le -r\cdot\frac{\epsilon}{2}+r\cdot\frac{1}{2/\epsilon-1}\cdot\left(1-\frac{\epsilon}{2}\right)\\ &= -r\cdot\frac{\epsilon}{2}+r\cdot\frac{\epsilon}{2}\le 0, \end{aligned}$$

giving the required bound. This proves (4.4) and completes the theorem. □

# 5 Matching lower bound

Here we show that the row count obtained by Theorem 4.1 is in fact optimal. While it is possible to obtain a spectral approximation with $\mathcal{O}(d/\epsilon^2)$ rows in the offline setting, online sampling always incurs a loss of $\Omega\left(\log(\epsilon\|\mathrm{A}\|_2^2/\delta)\right)$ and must sample $\Omega\left(d\log(\epsilon\|\mathrm{A}\|_2^2/\delta)/\epsilon^2\right)$ rows.

Theorem 5.1. Assume that $\epsilon\|\mathrm{A}\|_2^2 \geq c_1\delta$ and $\epsilon \geq c_2/\sqrt{d}$, for fixed constants $c_1$ and $c_2$. Then any algorithm that selects rows in an online manner and outputs a spectral approximation to $\mathrm{A}^T\mathrm{A}$ with $(1+\epsilon)$ multiplicative error and $\delta$ additive error with probability at least 1/2 must sample $\Omega\left(d\log(\epsilon\|\mathrm{A}\|_2^2/\delta)/\epsilon^2\right)$ rows of A in expectation.

Note that the lower bounds we assume on $\epsilon\|\mathrm{A}\|_2^2$ and $\epsilon$ are very minor. They just ensure that $\log(\epsilon\|\mathrm{A}\|_2^2/\delta) \geq 1$ and that $\epsilon$ is not so small that we can essentially sample all rows.

Proof. We apply Yao's minimax principle, constructing, for any large enough $M$, a distribution on inputs A with $\|\mathrm{A}\|_2^2 \leq M$ for which any deterministic online row selection algorithm that succeeds with probability at least 1/2 must output $\Omega\left(d\log(\epsilon M/\delta)/\epsilon^2\right)$ rows in expectation. The best randomized algorithm that works with probability 1/2 on any input matrix with $\|\mathrm{A}\|_2^2 \leq M$ therefore must select at least $\Omega\left(d\log(\epsilon M/\delta)/\epsilon^2\right)$ rows in expectation on the worst case input, giving us the theorem.

Our distribution is as follows. We select an integer $N$ uniformly at random from $[1, \log(M\epsilon/\delta)]$. We then stream in the vertex-edge incidence matrices of $N$ complete graphs on $d$ vertices. We double the weight of each successive graph. Intuitively, spectrally approximating a complete graph requires selecting $\Omega(d/\epsilon^2)$ edges [2] (as long as $\epsilon \geq c_2/\sqrt{d}$ for some fixed constant $c_2$). Each time we stream in a new graph with double the weight, we force the algorithm to add $\Omega(d/\epsilon^2)$ more edges to its output, eventually forcing it to return $\Omega(d/\epsilon^2 \cdot N)$ edges, which is $\Omega(d\log(M\epsilon/\delta)/\epsilon^2)$ in expectation.

Specifically, let $\mathrm{K}_d$ be the $\binom{d}{2} \times d$ vertex-edge incidence matrix of the complete graph on $d$ vertices. $\mathrm{K}_d^T\mathrm{K}_d$ is the Laplacian matrix of the complete graph on $d$ vertices. We weight the first graph so that its Laplacian has all its nonzero eigenvalues equal to $\delta/\epsilon$. (That is, each edge has weight $\delta/(d\epsilon)$). In this way, even if we select $N = \lfloor\log(M\epsilon/\delta)\rfloor$ we have overall $\|\mathrm{A}\|_2^2 \leq \delta/\epsilon + 2\delta/\epsilon + \cdots + 2^{\lfloor\log(M\epsilon/\delta)\rfloor - 1}\delta/\epsilon \leq M$.

Even if $N = 1$, all nonzero eigenvalues of $\mathrm{A}^T\mathrm{A}$ are at least $\delta/\epsilon$, so achieving $(1+\epsilon)$ multiplicative error and $\delta\mathrm{I}$ additive error is equivalent to achieving $(1+2\epsilon)$ multiplicative error. $\mathrm{A}^T\mathrm{A}$ is a graph Laplacian so has a null space. However, as all rows are orthogonal to the null space, achieving additive error $\delta\mathrm{I}$ is equivalent to achieving additive error $\delta\mathrm{I}_r$ where $\mathrm{I}_r$ is the identity projected to the span of $\mathrm{A}^T\mathrm{A}$. $\delta\mathrm{I}_r \preceq \epsilon\mathrm{A}^T\mathrm{A}$ which is why we must achieve $(1+2\epsilon)$ multiplicative error.

In order for a deterministic algorithm to be correct with probability 1/2 on our distribution, it must be correct for at least 1/2 of our $\lfloor\log(M\epsilon/\delta)\rfloor$ possible choices of $N$.

Let $i$ be the lowest choice of $N$ for which the algorithm is correct. By the lower bound of [2], the algorithm must output $\Omega(d/\epsilon^2)$ rows of $\mathrm{A}_i$ to achieve a $(1+2\epsilon)$ multiplicative-error spectral approximation. Here $\mathrm{A}_i$ is the input consisting of the vertex-edge incidence matrices of $i$ increasingly weighted complete graphs. Call the output on this input $\text{Ã}_i$. Now let $j$ be the second lowest choice of $N$ on which the algorithm is correct. Since the algorithm was correct on $\mathrm{A}_i$ to within a multiplicative $(1+2\epsilon)$, to be correct on $\mathrm{A}_j$, it must output a set of edges $\text{Ã}_j$ such that

$$(\mathrm{A}_j^T\mathrm{A}_j - \mathrm{A}_i^T\mathrm{A}_i) - 4\epsilon\mathrm{A}_j^T\mathrm{A}_j \preceq \text{Ã}_j^T\text{Ã}_j - \text{Ã}_i^T\text{Ã}_i \preceq (\mathrm{A}_j^T\mathrm{A}_j - \mathrm{A}_i^T\mathrm{A}_i) + 4\epsilon\mathrm{A}_j^T\mathrm{A}_j\,.$$

Since we double each successive copy of the complete graph, $\mathrm{A}_j^T\mathrm{A}_j \preceq 2(\mathrm{A}_j^T\mathrm{A}_j - \mathrm{A}_i^T\mathrm{A}_i)$. So, $\text{Ã}_j^T\text{Ã}_j - \text{Ã}_i^T\text{Ã}_i$ must be a $1+8\epsilon$ spectral approximation to the true difference $\mathrm{A}_j^T\mathrm{A}_j - \mathrm{A}_i^T\mathrm{A}_i$. Noting that this difference is itself just a weighting of the complete graph, by the lower bound in [2] the algorithm must select $\Omega(d/\epsilon^2)$ additional edges between the $i^{th}$ and $j^{th}$ input graphs. Iterating this argument over all $\lfloor\log(M\epsilon/\delta)\rfloor/2$ inputs on which the algorithm must be correct, it must select a total of $\Omega(d\log(M\epsilon/\delta)/\epsilon^2)$ edges in expectation over all inputs. □

# 6 Future work

The main open question arising from the original publication of this work [13] was if one could prove that the algorithm of [16] works despite dependencies arising due to the row pruning step. By operating

in the online setting, our algorithm avoids row pruning, and hence is able to skirt these dependencies, as the probability that a row is sampled only depends on earlier rows in the stream. However, because the streaming setting offers the potential for sampling fewer rows than in the online case, obtaining a rigorous proof of [16] is very interesting. This open question was essentially resolved in [18], which presents an algorithm similar to the one presented in [16] for insertion-only streams that admits a correct proof.

While our work focuses on spectral approximation, variants on (ridge) leverage score sampling and the BSS algorithm are also used to solve low-rank approximation problems, including column subset selection [5, 12] and projection-cost-preserving sketching [10, 12]. Compared with spectral approximation, there is less work on streaming sampling for low-rank approximation, and understanding how online algorithms may be used in this setting would an interesting directino. Since initial publication, this question has been studied extensively [6, 8, 7], with online ridge leverage scores being employed for online low-rank approximation of kernel matrices and for low-rank approximation in sliding window streams.

Acknowledgements. The authors would like to thank Kenneth Clarkson, Jonathan Kelner, Gary Miller, Christopher Musco and Richard Peng for helpful discussions and comments. Cameron Musco and Jakub Pachocki both acknowledge the Gene Golub SIAM Summer School program on Randomization in Numerical Linear Algebra, where work on this project was initiated.

## References

[1] Ahmed Alaoui and Michael W. Mahoney: Fast randomized kernel ridge regression with statistical guarantees. In Adv. Neural Info. Proc. Sys. 30 (NIPS'15), pp. 775–783. Curran Assoc., Inc., 2015. NIPS. [arXiv:1411.0306]

[2] Joshua Batson, Daniel A. Spielman, and Nikhil Srivastava: Twice-Ramanujan sparsifiers. SIAM J. Comput., 41(6):1704–1721, 2012. Preliminary version in STOC'09. [doi:10.1137/090772873, arXiv:0808.0163]

[3] Antoine Bordes and Léon Bottou: The huller: a simple and efficient online SVM. In Machine Learning: ECML'05, pp. 505–512. Springer, 2005. [doi:10.1007/11564096_48]

[4] Christos Boutsidis, Dan Garber, Zohar Karnin, and Edo Liberty: Online principal components analysis. In Proc. 26th Ann. ACM-SIAM Symp. on Discrete Algorithms (SODA'15), pp. 887–901. ACM Press, 2015. [doi:10.1137/1.9781611973730.61]

[5] Christos Boutsidis and David P. Woodruff: Optimal CUR matrix decompositions. SIAM J. Comput., 46(2):543–589, 2017. Preliminary version in STOC'14. [arXiv:1405.7910]

[6] Vladimir Braverman, Petros Drineas, Cameron Musco, Christopher Musco, Jalaj Upadhyay, David P. Woodruff, and Samson Zhou: Numerical linear algebra in the online and sliding window models. In Proc. 61st FOCS, pp. 517–528. IEEE Comp. Soc., 2020. [doi:10.1109/FOCS46700.2020.00055, arXiv:1805.03765]

[7] Daniele Calandriello, Alessandro Lazaric, and Michal Valko: Efficient second-order online kernel learning with adaptive embedding. In Adv. Neural Info. Proc. Sys. 30 (NIPS'17), pp. 6140–6150. Curran Assoc., Inc., 2017. NIPS.

[8] Daniele Calandriello, Alessandro Lazaric, and Michal Valko: Second-order kernel online convex optimization with adaptive sketching. In Proc. 34th Int. Conf. on Machine Learning (ICML'17), pp. 645–653, 2017. MLR Press. [arXiv:1706.04892]

[9] Kenneth L. Clarkson and David P. Woodruff: Low rank approximation and regression in input sparsity time. J. ACM, 63(6):54:1–54:45, 2017. Preliminary version in in STOC'13. [doi:10.1145/3019134, arXiv:1207.6365]

[10] Michael B. Cohen, Sam Elder, Cameron Musco, Christopher Musco, and Madalina Persu: Dimensionality reduction for $k$-means clustering and low rank approximation. In Proc. 47th STOC, pp. 163–172. ACM Press, 2015. [doi:10.1145/2746539.2746569, arXiv:1410.6801]

[11] Michael B. Cohen, Yin Tat Lee, Cameron Musco, Christopher Musco, Richard Peng, and Aaron Sidford: Uniform sampling for matrix approximation. In Proc. 6th Innovations in Theoret. Comp. Sci. conf. (ITCS'15), pp. 181–190. ACM Press, 2015. [doi:10.1145/2688073.2688113, arXiv:1408.5099]

[12] Michael B. Cohen, Cameron Musco, and Christopher Musco: Input sparsity time low-rank approximation via ridge leverage score sampling. In Proc. 28th Ann. ACM-SIAM Symp. on Discrete Algorithms (SODA'17), pp. 1758–1777. ACM Press, 2017. [doi:10.1137/1.9781611974782.115, arXiv:1511.07263]

[13] Michael B. Cohen, Cameron Musco, and Jakub Pachocki: Online row sampling. In Proc 19th Internat. Workshop on Approximation Algorithms for Combinat. Opt. Probl. (APPROX'16), pp. 7:1–7:18. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2016. [doi:10.4230/LIPIcs.APPROX-RANDOM.2016.7]

[14] Koby Crammer, Ofer Dekel, Joseph Keshet, Shai Shalev-Shwartz, and Yoram Singer: Online passive-aggressive algorithms. J. Mach. Learning Res., 7:551–585, 2006. JMLR.

[15] Michael Kapralov, Yin Tat Lee, Cameron Musco, Christopher Musco, and Aaron Sidford: Single pass spectral sparsification in dynamic streams. SIAM J. Comput., 46(1):456–477, 2017. Preliminary version in in FOCS'14. [doi:10.1137/141002281, arXiv:1407.1289]

[16] Jonathan A. Kelner and Alex Levin: Spectral sparsification in the semi-streaming setting. Theory Comput. Sys., 53(2):243–262, 2013. [doi:10.1007/s00224-012-9396-1]

[17] Ioannis Koutis, Gary L. Miller, and Richard Peng: Approaching optimality for solving SDD linear systems. SIAM J. Comput., 43(1):337–354, 2014. Preliminary version in in FOCS'10. [doi:10.1137/110845914, arXiv:1003.2958]

[18] Rasmus Kyng, Jakub Pachocki, Richard Peng, and Sushant Sachdeva: A framework for analyzing resparsification algorithms. In Proc. 28th Ann. ACM-SIAM Symp. on Discrete Algorithms (SODA'17), pp. 2032–2043. ACM Press, 2017. [doi:10.1137/1.9781611974782.132, arXiv:1611.06940]

[19] Yin Tat Lee and He Sun: Constructing linear-sized spectral sparsification in almost-linear time. SIAM J. Comput., 47(6):2315–2336, 2018. Preliminary version in in FOCS'15. [doi:10.1137/16M1061850, arXiv:1508.03261]

[20] Mu Li, Gary L. Miller, and Richard Peng: Iterative row sampling. In Proc. 54th FOCS, pp. 127–136. IEEE Comp. Soc., 2013. [doi:10.1109/FOCS.2013.22, arXiv:1211.2713]

[21] Edo Liberty, Ram Sriharsha, and Maxim Sviridenko: An algorithm for online k-means clustering. In Proc. 18th Workshop on Algorithm Engineering and Experiments (ALENEX'16), pp. 81–89, 2016. [doi:10.1137/1.9781611974317.7, arXiv:1412.5721]

[22] Xiangrui Meng and Michael W. Mahoney: Low-distortion subspace embeddings in input-sparsity time and applications to robust linear regression. In Proc. 45th STOC, pp. 91–100. ACM Press, 2013. [doi:10.1145/2488608.2488621, arXiv:1210.3135]

[23] Jelani Nelson and Huy Lê Nguyễn: OSNAP: Faster numerical linear algebra algorithms via sparser subspace embeddings. In Proc. 54th FOCS, pp. 117–126. IEEE Comp. Soc., 2013. [doi:10.1109/FOCS.2013.21, arXiv:1211.1002]

[24] Jack Sherman and Winifred J. Morrison: Adjustment of an inverse matrix corresponding to a change in one element of a given matrix. Ann. Math. Stat., 21(1):124–127, 1950. JSTOR.

[25] Daniel A. Spielman and Nikhil Srivastava: Graph sparsification by effective resistances. SIAM J. Comput., 40(6):1913–1926, 2011. Preliminary version in in STOC'08. [doi:10.1137/080734029, arXiv:0803.0929]

[26] Daniel A. Spielman and Shang-Hua Teng: Nearly-linear time algorithms for graph partitioning, graph sparsification, and solving linear systems. In Proc. 36th STOC, pp. 81–90. ACM Press, 2004. [doi:10.1145/1007352.1007372, arXiv:cs/0310051]

[27] Daniel A. Spielman and Shang-Hua Teng: Nearly linear time algorithms for preconditioning and solving symmetric, diagonally dominant linear systems. SIAM J. Matrix Anal. Appl., 35(3):835–885, 2014. [doi:10.1137/090771430]

[28] Joel A. Tropp: Freedman's inequality for matrix martingales. Electr. Comm. Probability, 16(25):262–270, 2011. [doi:10.1214/ECP.v16-1624, arXiv:1101.3039]